

Data Wrangling 🤠

Reading CSV Files

Lesson 15

Where We Left Off

- Last lesson we built tables **by hand** — typing out a list of dictionaries, or a dictionary of lists, ourselves
- Real data doesn't start out as Python code — it lives in **files**
- Today: read a real table off of disk, into the structures we just built

What Is a CSV?

- **CSV** = **C**omma-**S**eparated **V**alues — a plain text format for tables
- One line per row; values on a line are separated by commas
- The **first line** is usually a **header**: the column names

```
State,Region,Capital City,Population,Land Area,Party  
Alabama,South,Montgomery,5024279,50645,Republican  
Alaska,West,Juneau,733391,570641,Republican  
Arizona,West,Phoenix,7151502,113594,Republican
```

- It's just text! You could open `states.csv` in a text editor and read it yourself

Opening a File in Python

- `open(filename)` gives you a **file object** you can read from
- Use `with open(...) as f:` so Python automatically **closes** the file when you're done with it

```
with open("states.csv") as f:  
    for line in f:  
        print(line)
```

- Everything inside the `with` block can use `f`; once you leave the block, the file is closed automatically

Reading Raw Lines

```
with open("states.csv") as f:  
    for line in f:  
        print(line)
```

State,Region,Capital City,Population,Land Area,Party

Alabama,South,Montgomery,5024279,50645,Republican

Alaska,West,Juneau,733391,570641,Republican

- Each `line` is just **one long string** — commas and all
- To pull out individual fields, we'd have to call `line.split(",")` ourselves... for every line, in every file, forever

The `csv` Module

- Python comes with a built-in `csv` module that handles all of that for us
- `import csv` at the top of your file to use it
- Two tools we'll use today:
 - `csv.reader` → each row as a **list** of strings
 - `csv.DictReader` → each row as a **dictionary** of strings

```
import csv
```

Demo: `csv.reader`

```
import csv

with open("states.csv") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

```
['State', 'Region', 'Capital City', 'Population', 'Land Area', 'Party']
['Alabama', 'South', 'Montgomery', '5024279', '50645', 'Republican']
['Alaska', 'West', 'Juneau', '733391', '570641', 'Republican']
```

- Every row — **including the header** — comes back as a `list` of strings

Demo: `csv.DictReader`

```
import csv

with open("states.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row)
```

```
{'State': 'Alabama', 'Region': 'South', 'Capital City': 'Montgomery',
 'Population': '5024279', 'Land Area': '50645', 'Party': 'Republican'}
{'State': 'Alaska', 'Region': 'West', 'Capital City': 'Juneau',
 'Population': '733391', 'Land Area': '570641', 'Party': 'Republican'}
```

- `DictReader` automatically uses the **first line** as keys
- Every row comes back as a **dictionary** — exactly the row-oriented

Building the List of Dictionaries

Combine `DictReader` with an empty list and `.append` — the exact pattern from Lesson 11 for building up a list in a loop:

```
import csv

states = []
with open("states.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        states.append(row)

>>> states[0]
{'State': 'Alabama', 'Region': 'South', 'Capital City': 'Montgomery', ...}
>>> states[0]["Population"]
'5024279'
```



Everything Is a String!

```
>>> states[0]["Population"]  
'5024279'  
>>> type(states[0]["Population"])  
<class 'str'>
```

- `csv.DictReader` reads **every value as text** — even ones that look like numbers
- Try to do math with it, and you won't get what you expect:

```
>>> states[0]["Population"] + 1  
TypeError: can only concatenate str (not "int") to str
```

Converting Types Yourself

- Use `int(...)` or `float(...)` to convert a field before doing math with it

```
>>> population = int(states[0]["Population"])
>>> population
5024279
>>> population + 1
5024280
```

- You have to do this **field by field** — Python doesn't know which columns are numeric

```
row = states[0]
```

Concept Check

Given a freshly-read row

`row = {"State": "Texas", "Population": "29145505"}`, what is the value and type of each expression?

```
row["Population"]
```

```
row["Population"] * 2
```

```
int(row["Population"]) * 2
```

Putting It All Together

```
import csv

def read_states(filename):
    rows = []
    with open(filename) as f:
        reader = csv.DictReader(f)
        for row in reader:
            row["Population"] = int(row["Population"])
            row["Land Area"] = int(row["Land Area"])
            rows.append(row)
    return rows

states = read_states("states.csv")
>>> states[0]["Population"] / states[0]["Land Area"]
99.21...
```

Back to Column-Oriented

Once you have a row-oriented table, converting to column-oriented is the *exact* same loop from last lesson — the data just happens to have come from a file this time:

```
columns = {"State": [], "Population": []}

for row in states:
    columns["State"].append(row["State"])
    columns["Population"].append(row["Population"])

>>> sum(columns["Population"]) / len(columns["Population"])
6506226.66...
```

Summary

- A **CSV** is a plain text table — one line per row, commas between values, header on line one
- `open(...)` / `with` gives you access to a file; `import csv` gives you tools to parse it
- `csv.DictReader` reads each row as a **dictionary**, using the header as keys — row-oriented, for free
-  Every value comes back as a **string** — convert numeric fields yourself with `int(...)` / `float(...)`

Up Next